Netskope Client Deployment & Traffic Steering at Scale (Intune)

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Netskope Client Deployment & Traffic Steering at Scale (Intune) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Netskope Client Deployment & Traffic Steering at Scale (Intune) 5

What this solution is . . . 5

Why it matters (the why lives here) . . . 5

How it works . . . 5

What you configure (named artifacts, reused verbatim throughout) . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 7

2. Requirements Guide — Netskope Client Deployment & Traffic Steering at Scale (Intune) 7

2.1 Netskope client installer and tenant-enrollment method . . . 7

2.2 Steering configuration authored in the Netskope tenant . . . 7

2.3 Exception list for business applications . . . 7

2.4 SSL/decryption certificate delivery (if SSL inspection is used) . . . 7

2.5 Devices enrolled and compliant in Intune . . . 8

2.6 Rollout-ring groups (pilot first, then larger waves) . . . 8

2.7 Change window and rollback path . . . 8

Checklist . . . 8

3. Implementation Guide — Netskope Client Deployment & Traffic Steering at Scale (Intune) 8

Step 1 — Assess readiness . . . 9

Step 2 — Create the ring groups . . . 9

Step 3 — Package the Netskope client as an Intune app . . . 9

Step 4 — Deliver the SSL/decryption certificate (if inspecting SSL) . . . 10

Step 5 — Confirm steering and exception configuration (Netskope tenant) . . . 10

Step 6 — Deploy to the pilot ring . . . 10

Step 7 — Validate the pilot ring (pilot gate) . . . 10

Step 8 — Expand to the early ring . . . 11

Step 9 — Expand to broad (production) . . . 11

Step 10 — Steady-state operations . . . 11

5. Testing Guide — Netskope Client Deployment & Traffic Steering at Scale (Intune) 11

Test 1 — App deployment succeeded . . . 11

Test 2 — Client present and healthy on device . . . 12
Test 3 — Steering is actually moving traffic . . . 12
Test 4 — Exceptions keep business apps working . . . 12
Test 5 — SSL inspection trusts cleanly (if used) . . . 12
Test 6 — Fleet deployment + exception coverage report . . . 12
Test 7 — Ring targeting is correct . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — Netskope Client Deployment & Traffic Steering at Scale (Intune) 13
A. Client not installing via Intune . . . 13
B. Client installed but not healthy . . . 13
C. Steering breaks a business app . . . 13
D. Certificate / SSL inspection errors . . . 14
E. Some devices uncovered after a wave . . . 14
F. Rolling back (live-traffic safety) . . . 14
G. When to escalate . . . 14
7. Best Practices — Netskope Client Deployment & Traffic Steering at Scale (Intune) 14
8. Deliverables Summary — Netskope Client Deployment & Traffic Steering at Scale (Intune) 15
Documentation . . . 15
PowerShell scripts . . . 15
What the buyer receives . . . 16
Outcome . . . 16
Appendix A — PowerShell Scripts 17

Executive Summary — Netskope Client Deployment & Traffic Steering at Scale (Intune)

A SASE platform only protects what its client actually reaches. If the Netskope client is missing, unhealthy, or steering the wrong traffic, the whole investment underdelivers — and done by hand, agent rollouts stall: clients fail to enroll, steering configs drift between machines, and exceptions break business applications. This solution gives you the repeatable way to deploy the Netskope client and traffic steering at scale through Microsoft Intune: packaged deployment, steering and exception configuration, client-health validation, and a ringed rollout to thousands of endpoints — proven on a pilot before you scale.

This is a complete, deploy-it-today package, written for an administrator with no prior agent-rollout experience. Netskope console internals are deferred to the official Netskope documentation; the focus here is the Intune-based packaging, deployment, steering and exception wiring, health checks, and the ringed large-scale rollout you own.

What you get

• A plain-language Solution Overview of the Netskope client, the steering model, exceptions, client health, and the ringed rollout — and the why behind each.

• A Requirements Guide with Description, Why, Verify, and Implement-if-missing for every prerequisite.

• An Implementation Guide of precise, ordered Intune steps: package and deploy the client, deliver steering and exception config, validate health, build rollout rings, then pilot, validate, expand.

• Five PowerShell scripts for client-readiness discovery, client-health and steering validation, a deployment-status and exception report, safe remediation of failed clients, and a rollout-ring group builder.

• Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

A packaged Netskope client deployed through Intune, consistent steering and an exception list that keeps business apps working, validated client health across the fleet, and a ringed rollout — a small pilot ring first, then progressively larger rings — with a pilot gate and a documented rollback path before any wider expansion.

The standard deployment model

Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

The Netskope client on every device, healthy and steering correctly — deployed through Intune in safe rings, with exceptions that do not break business apps, and validated on a pilot before you scale.

1. Solution Overview — Netskope Client Deployment & Traffic Steering at Scale (Intune)

What this solution is

A SASE/SSE platform enforces policy in the cloud, but traffic only reaches that cloud if an endpoint agent — the Netskope client — is installed, healthy, enrolled to your tenant, and steering the right traffic. This solution is the repeatable way to deliver that agent and its steering configuration to a large managed fleet using Microsoft Intune: you package the client once, deploy it through Intune, push consistent steering and exception configuration, validate client health, and roll out in safe rings to thousands of endpoints.

The Netskope console internals (tenant settings, steering-profile screens, exception object names) change over time and are deferred to the official Netskope documentation and your tenant. What this solution owns end to end is the Intune side: packaging, deployment, assignment, health validation, and the ringed large-scale rollout.

Why it matters (the why lives here)

- Coverage is the whole game. A SASE policy that a device never touches is no policy at all; the client must be on every in-scope endpoint, healthy, and enrolled to your tenant.
- Hand rollouts stall. Manual installs miss machines, apply inconsistent steering, and accumulate one-off exceptions nobody can audit — leaving silent coverage gaps.
- Steering and exceptions affect live traffic. Inline steering and SSL inspection sit in the path of real user traffic; a wrong exception (or a missing one) either breaks a business app or quietly bypasses protection. Both must be deliberate and consistent.
- Scale needs rings. Pushing a traffic-steering agent to thousands of devices at once is how an outage happens. A small pilot ring proves health and steering before each larger ring inherits it.

How it works

1. The Netskope client installer is wrapped as an Intune app (the client app artifact), configured for unattended install with your tenant-enrollment method, and assigned to device groups.
1. Steering and exception configuration is delivered so every device steers the same traffic and honors the same exception list — applied consistently rather than per-machine.
1. Client health is validated on each device (installed, running, enrolled to the right tenant, steering active) and reported across the fleet from Intune.
1. Rollout is ringed: a small pilot ring installs first, then progressively larger rings, each gated on the prior ring's health and steering being clean.
1. Deployment status, failures, and exception coverage are reported so laggards and broken clients

are found and remediated rather than left silently unprotected.

What you configure (named artifacts, reused verbatim throughout)

- Client app: `Netskope Client – Windows – Production` — the packaged Intune app that installs and

tenant-enrolls the client.

- Steering config: `Steering Profile – Corporate – Production` — the traffic-steering definition

delivered to managed devices (authored in the Netskope tenant; deferred to Netskope docs).

- Exception list: `Steering Exceptions – Business Apps – Production` — the bypass/exception set that

keeps named business applications working under inline steering.

- Rollout rings: `Rollout Ring – Pilot – Stage1`, `Rollout Ring – Early – Stage2`,

`Rollout Ring – Broad – Stage3` — the progressively larger waves of the deployment.

- Groups: `Device Group – Pilot – Stage1`, `Device Group – Early – Stage2`,

`Device Group – Broad – Stage3` — the Entra security groups each ring is assigned to.

Benefits

- Coverage at scale — the client lands on every in-scope device through Intune, not by hand.
- Consistent steering — one steering config and one exception list applied fleet-wide, not per machine.
- Apps keep working — a deliberate, named exception list prevents inline steering from breaking

business apps.

- Safe rollout — ringed waves with a pilot gate catch problems on a few devices, not thousands.
- Visibility and recovery — per-device deployment status and health, with safe remediation for

failed or unhealthy clients.

Limitations (be honest with buyers)

- Netskope tenant configuration (steering profiles, exception objects, enrollment method, SSL/decrypt

certificates) is authored in the Netskope console and is deferred to the official Netskope documentation — this solution wires the Intune delivery and validation around it.

- The client packaging specifics (silent-install switches, enrollment token/SSO method) are

vendor-version-dependent; the steps name what to set and defer the exact switches to Netskope docs.

- Steering changes affect live traffic; a change window and rollback path are required (global G7).
- Client-health validation reads device-side state and Intune deployment status; deep steering

analytics live in the Netskope console.

The standard deployment model

> *Package and stage the client app -> deliver steering + exception config -> validate client health -> ring the rollout -> pilot, validate, expand. The model: Assess -> Configure -> Pilot -> Validate -> Expand.*

2. Requirements Guide — Netskope Client Deployment & Traffic Steering at Scale (Intune)

Verify every requirement before configuring. Each entry: Description, Why, Verify, Implement-if-missing. Category-wide prerequisites (Netskope tenant, Entra ID, Intune, admin roles, network/identity fundamentals, pilot group, change control) are in `global-requirements-network.md` — confirm those first; the items below are specific to this client-deployment-and-steering solution.

2.1 Netskope client installer and tenant-enrollment method

Description: The current Netskope client installer plus the tenant-enrollment method your tenant uses (for example an enrollment token or identity/SSO-based enrollment). Why: The Intune app must install the client silently and enroll it to the correct Netskope tenant, or devices install an unconfigured client that steers nothing. Verify: You have the installer and the documented silent-install and enrollment parameters from your Netskope tenant; a manual install on one test device installs and enrolls to your tenant. Implement: Obtain the installer and enrollment method from your Netskope tenant/account team; confirm the exact silent-install switches in the official Netskope documentation (deferred to Netskope docs).

2.2 Steering configuration authored in the Netskope tenant

Description: A steering profile (`Steering Profile – Corporate – Production`) defined in the Netskope tenant that specifies which traffic the client steers. Why: The client steers according to tenant-side configuration; without it, deployment installs an agent with no defined steering behavior. Verify: In your Netskope tenant, the steering profile exists and is assigned to your users/devices (exact console path deferred to Netskope docs). Implement: Author the steering profile in the Netskope console per the official Netskope documentation.

2.3 Exception list for business applications

Description: A defined set of bypass/steering exceptions (`Steering Exceptions – Business Apps – Production`) for applications that must not be inline-steered or SSL-inspected. Why: Inline steering and decryption can break apps that pin certificates or dislike proxies; a named, auditable exception list keeps business apps working without disabling protection broadly. Verify: The exception entries for your known sensitive apps exist in the tenant configuration; a test device with the client active can still use those apps. Implement: Build the exception list in the Netskope console (objects/format deferred to Netskope docs); record each entry and the app it protects so it is auditable.

2.4 SSL/decryption certificate delivery (if SSL inspection is used)

Description: If the steering config inspects SSL, the Netskope root/decryption certificate delivered to the device trust store via Intune. Why: Without the trusted certificate, inspected HTTPS traffic throws certificate

errors fleet-wide. Verify: On a test device the certificate is present in the trusted root store and inspected sites load without certificate warnings. Implement: Deploy the certificate with an Intune trusted-certificate profile to the same device groups as the client (certificate source/details deferred to Netskope docs).

2.5 Devices enrolled and compliant in Intune

Description: The in-scope Windows devices enrolled in Intune and (where used) compliant. Why: Intune app and certificate assignments apply only to enrolled devices. Verify: Intune -> Devices -> All devices shows the target devices; run `NSCLIENT-Discovery.ps1`. Implement: Enable automatic enrollment and enroll the target devices (global G3).

2.6 Rollout-ring groups (pilot first, then larger waves)

Description: Entra security groups for each ring — `Device Group – Pilot – Stage1`, `Device Group – Early – Stage2`, `Device Group – Broad – Stage3`. Why: A ringed rollout requires the membership boundaries each wave is assigned to; the pilot ring is the gate before any expansion. Verify: Entra -> Groups shows the ring groups; run `NSCLIENT-Set-RolloutRingTargeting.ps1 -WhatIf` to preview membership. Implement: Create the groups in Entra (or build them with `NSCLIENT-Set-RolloutRingTargeting.ps1`); keep the pilot small and representative.

2.7 Change window and rollback path

Description: An approved change window and a written rollback (disable steering, remove the client app assignment, or revert the exception list) before enabling steering on real traffic. Why: Steering and decryption sit in the live traffic path and must be reversible (global G7). Verify: A rollback procedure is documented and approved before the pilot ring goes inline. Implement: Define and approve the rollback and a user-comms plan (global G7).

Checklist

- Netskope client installer + tenant-enrollment method (Netskope docs)
- Steering Profile – Corporate – Production authored in the tenant
- Steering Exceptions – Business Apps – Production defined
- SSL/decryption certificate delivery configured (if inspecting SSL)
- Target devices enrolled/compliant in Intune — `NSCLIENT-Discovery.ps1`
- Ring groups created — `NSCLIENT-Set-RolloutRingTargeting.ps1 -WhatIf`
- Change window + rollback path approved

3. Implementation Guide — Netskope Client Deployment & Traffic Steering at Scale (Intune)

Follow these steps in order, in the Microsoft Intune admin center (`https://intune.microsoft.com`). Netskope tenant-side configuration (the steering profile, exception objects, enrollment method, and

SSL/decryption certificate) is authored in the Netskope console and is deferred to the official Netskope documentation; this guide wires the Intune delivery, validation, and ringed rollout around it.

> *Model: Assess the fleet, configure the client app and steering/exception delivery, deploy to a small pilot ring, validate health and steering, then expand ring by ring. A pilot gate sits before every expansion, and a rollback path is ready before steering touches live traffic.*

Step 1 — Assess readiness

1. Run `NSCLIENT-Discovery.ps1` on representative devices to confirm OS, enrollment, and that no prior/partial Netskope client is present.
1. Run `NSCLIENT-Set-RolloutRingTargeting.ps1 -WhatIf` to preview the ring groups and membership.
2. Confirm the Netskope tenant has `Steering Profile – Corporate – Production` and `Steering Exceptions – Business Apps – Production` ready (Requirements 2.2-2.3).

Step 2 — Create the ring groups

1. Go to Entra -> Groups -> New group -> Security -> name `Device Group – Pilot – Stage1` (a few representative devices across your hardware/app profiles). Select Create.
1. Repeat for `Device Group – Early – Stage2` (a larger cross-section) and `Device Group – Broad – Stage3` (the remaining fleet).
1. Alternatively, build these groups with `NSCLIENT-Set-RolloutRingTargeting.ps1` (run with -WhatIf first, then re-run without it to apply).

Step 3 — Package the Netskope client as an Intune app

1. Wrap the Netskope client installer using the Microsoft Win32 Content Prep Tool to produce a `.intunewin` package (use the silent-install and tenant-enrollment switches from the Netskope documentation — deferred to Netskope docs).
1. Go to Intune -> Apps -> Windows -> Add -> app type Windows app (Win32) -> select the `.intunewin`.
2. Name the app `Netskope Client – Windows – Production`. Complete the publisher/description fields.
3. Program: enter the silent install command and the uninstall command from the Netskope documentation; set the install behavior to System.
1. Detection rules: add a rule that reliably detects the installed client (for example the client service or install path/version per the Netskope documentation).
1. Select Next through Requirements/Dependencies; do not assign yet. Select Create.

Step 4 — Deliver the SSL/decryption certificate (if inspecting SSL)

1. Go to Intune -> Devices -> Configuration -> Create -> New policy -> Windows -> profile type

Templates -> Trusted certificate.

1. Name it `Netskope Decryption Cert – Trusted Root – Production`, upload the Netskope root/decryption

certificate (source deferred to Netskope docs), and set the destination store to Trusted Root.

1. Assignments: assign to `Device Group – Pilot – Stage1` for now (you will widen it with each ring).

Select Create.

Step 5 — Confirm steering and exception configuration (Netskope tenant)

1. In the Netskope tenant, confirm `Steering Profile – Corporate – Production` defines the traffic to

steer and is scoped to your pilot users/devices (console path deferred to Netskope docs).

1. Confirm `Steering Exceptions – Business Apps – Production` lists each sensitive business app and is

active on the steering profile.

1. Record which exception protects which app, so the list stays auditable.

Step 6 — Deploy to the pilot ring

1. Go to Intune -> Apps -> Windows -> open `Netskope Client – Windows – Production` -> Properties ->

Assignments -> Edit.

1. Under Required, add group `Device Group – Pilot – Stage1`. Save.

2. On a pilot device, sync policy (Settings -> Accounts -> Access work or school -> Info -> Sync), or

wait for the next Intune check-in.

Step 7 — Validate the pilot ring (pilot gate)

1. On pilot devices, run `NSCLIENT-Validation.ps1` -> confirms the client is installed, running,

enrolled to your tenant, and steering is active.

1. Run `NSCLIENT-Report.ps1` -> confirms the `Netskope Client – Windows – Production` deployment

status across `Device Group – Pilot – Stage1` (installed/failed/pending) and certificate coverage.

1. On a pilot device, confirm each business app on the exception list still works and that inspected

HTTPS sites load without certificate errors.

1. Complete the Testing Guide. Pilot gate: do not proceed to Step 8 until the pilot ring passes its pass/fail gate. If it fails, use `NSCLIENT-Remediation.ps1` (default -WhatIf) and the rollback path.

Step 8 — Expand to the early ring

1. Add `Device Group – Early – Stage2` to the Required assignment of `Netskope Client – Windows – Production`, and to the certificate profile from Step 4.
1. Widen the Netskope steering profile scope to the early-ring users/devices (Netskope docs).
2. Re-validate with `NSCLIENT-Validation.ps1` and `NSCLIENT-Report.ps1`; confirm exceptions still hold.

Step 9 — Expand to broad (production)

1. Add `Device Group – Broad – Stage3` to the app assignment and the certificate profile.
2. Widen the steering profile scope to the broad ring (Netskope docs).
3. Re-run `NSCLIENT-Report.ps1` each wave; chase failed/unhealthy clients with `NSCLIENT-Remediation.ps1`. Keep the rollback path available until broad is healthy.

Step 10 — Steady-state operations

1. Re-run `NSCLIENT-Report.ps1` on a schedule to catch new/failed/unhealthy clients.
2. Add new devices to the appropriate ring group (or to broad) so coverage stays complete.
3. Manage steering and exception changes through the same pilot-first ring path and change control.

Order of operations

```
Assess -> Ring groups -> Package client app -> Certificate (if SSL) -> Confirm steering + exceptions -> Deploy pilot -> Validate (pilot gate) -> Expand early -> Expand broad -> Operate
```

5. Testing Guide — Netskope Client Deployment & Traffic Steering at Scale (Intune)

Run after the pilot ring deploys (Implementation Step 7), and again after each ring expansion and any steering/exception change. Netskope console-side analytics are deferred to the Netskope documentation; these tests validate the Intune deployment and the device-side client/steering state.

Test 1 — App deployment succeeded

Do: Intune -> Apps -> Windows -> open `Netskope Client – Windows – Production` -> Device install status (for the ring under test). Expected: Devices in the ring show Installed; no Failed installs left unaddressed.

Test 2 — Client present and healthy on device

Do: On a ring device, run `NSCLIENT-Validation.ps1`. Expected: Client installed, service running, enrolled to the correct tenant, steering active.

Test 3 — Steering is actually moving traffic

Do: On a ring device, confirm the client reports steering active (per `NSCLIENT-Validation.ps1` and the client UI/logs defined by Netskope docs). Expected: In-scope traffic is steered; the device is not bypassing the platform.

Test 4 — Exceptions keep business apps working

Do: On a ring device with the client active, use each app on `Steering Exceptions – Business Apps – Production`. Expected: Every listed business app works; no app on the exception list is broken by steering/inspection.

Test 5 — SSL inspection trusts cleanly (if used)

Do: On a ring device, browse inspected HTTPS sites. Expected: Sites load with no certificate warnings (the decryption certificate is trusted via the Intune certificate profile).

Test 6 — Fleet deployment + exception coverage report

Do: Run `NSCLIENT-Report.ps1`. Expected: A clear per-ring breakdown of installed/failed/pending clients and certificate coverage; any gaps are visible and assigned for remediation.

Test 7 — Ring targeting is correct

Do: Run `NSCLIENT-Set-RolloutRingTargeting.ps1 -WhatIf`. Expected: Each ring group has the intended membership; no device is in two rings or missing from all.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-5 pass on `Device Group – Pilot – Stage1`; no business app broken; no cert errors
Early pass	Tests 1-6 pass on `Device Group – Early – Stage2` before broad expansion
Broad pass	Tests 1-6 pass on `Device Group – Broad – Stage3`; failed/unhealthy clients remediated

6. Troubleshooting Guide — Netskope Client Deployment & Traffic Steering at Scale (Intune)

Use `NSCLIENT-Discovery.ps1` and `NSCLIENT-Validation.ps1` for device state, `NSCLIENT-Report.ps1` for fleet deployment status, and `NSCLIENT-Remediation.ps1` (default -WhatIf) for safe fixes. Netskope console-side diagnostics (steering analytics, tenant policy state) are deferred to the official Netskope documentation.

A. Client not installing via Intune

Symptom	Cause	Remediation
App shows Failed	Wrong silent-install/uninstall command or bad detection rule	Re-check the install command and detection rule against Netskope docs; re-run `NSCLIENT-Discovery.ps1`
App stuck Pending	Device not synced or not in the ring group	Confirm group membership; sync the device; run `NSCLIENT-Remediation.ps1 -WhatIf` then apply
Installs but not detected	Detection rule does not match the installed client	Update the detection rule to the real install path/version/service per Netskope docs

B. Client installed but not healthy

Symptom	Cause	Remediation
Installed but not enrolled to tenant	Enrollment token/SSO method missing or wrong	Confirm the enrollment method (Netskope docs); reinstall with correct enrollment parameters
Service not running	Service stopped or blocked	Confirm the client service state with `NSCLIENT-Validation.ps1`; restart/reinstall via `NSCLIENT-Remediation.ps1`
Enrolled but not steering	Steering profile not scoped to this device	Confirm the device/user is in scope of `Steering Profile – Corporate – Production` (Netskope docs)

C. Steering breaks a business app

• Add the affected app to `Steering Exceptions – Business Apps – Production` (Netskope docs), confirm

the exception is active, then re-test on a pilot device before widening.

• For SSL pinning/cert errors specifically, see section D before assuming a steering issue.

D. Certificate / SSL inspection errors

Symptom	Cause	Remediation
Cert warnings on inspected sites	Decryption certificate not trusted on the device	Confirm the Intune trusted-certificate profile applied; widen its assignment to the device's ring group
One app fails TLS but others work	App pins certificates and cannot be inspected	Add the app to the steering/decryption exception list (Netskope docs)

E. Some devices uncovered after a wave

• Run `NSCLIENT-Report.ps1` to find devices with no/failed client; confirm they are in the correct ring group with `NSCLIENT-Set-RolloutRingTargeting.ps1 -WhatIf`, then sync and remediate.

F. Rolling back (live-traffic safety)

• To back out: remove the ring group from the `Netskope Client – Windows – Production` assignment,

and/or disable/narrow the steering profile scope in the Netskope tenant (global G7). Communicate the change and re-validate the remaining rings with `NSCLIENT-Validation.ps1`.

G. When to escalate

If the app deploys, the client is installed/running/enrolled, the device is synced and in scope, and steering still does not engage (or a non-excepted app still breaks), collect the client logs per the Netskope documentation and open a Netskope support case; for Intune delivery failures collect the Intune app install status and open a Microsoft support case.

7. Best Practices — Netskope Client Deployment & Traffic Steering at Scale (Intune)

• Always roll out in rings. A small `Device Group – Pilot – Stage1` proves the client, steering, and

exceptions before any larger ring inherits them — the single most important practice for an agent that sits in the live traffic path.

• Gate every expansion on the pilot. Treat the pilot pass/fail gate as mandatory; never widen the app

assignment or steering scope until the current ring is clean.

• One steering config, one exception list. Deliver `Steering Profile – Corporate – Production` and

`Steering Exceptions – Business Apps – Production` consistently fleet-wide; avoid per-machine steering tweaks that drift and cannot be audited.

• Build the exception list deliberately and keep it auditable. Record which exception protects which

business app; an undocumented exception is a silent coverage gap or a future broken app.

• Deliver the decryption certificate with the client. If you inspect SSL, assign the trusted-root certificate to the same ring groups as the app so HTTPS never breaks ahead of the client.

• Package for silent, unattended install. Use the vendor silent-install and tenant-enrollment switches (Netskope docs) and a reliable detection rule so Intune reports true install state.

• Validate health, not just install. A client can be installed but not enrolled or not steering; `NSCLIENT-Validation.ps1` checks running, enrolled, and steering — use it, don't assume.

• Keep a rollback path ready until broad is healthy. Disabling steering, pulling the app assignment, or reverting the exception list must be a known, approved move (global G7).

• Chase laggards every wave. Run `NSCLIENT-Report.ps1` on a schedule; a device with a failed or unhealthy client is an unprotected device.

• Remediate safely. Run `NSCLIENT-Remediation.ps1` in its default -WhatIf mode first to see what it would do before applying any change.

• Document the rollout. Record ring membership, steering config, exception entries, and certificate coverage for audits and repeatability.

8. Deliverables Summary — Netskope Client Deployment & Traffic Steering at Scale (Intune)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The Netskope client, steering model, exceptions, client health, ringed rollout, and the why
2. Requirements Guide	Every prerequisite with Description / Why / Verify / Implement-if-missing
Global Requirements (Network)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered Intune steps: package, deploy, certificate, steering/exception, ringed rollout
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`NSCLIENT-Discovery.ps1`	Reports device readiness: OS, Intune enrollment, and any prior/partial Netskope client (read-only)
`NSCLIENT-Validation.ps1`	Confirms the client is installed, running, enrolled, and steering active on the device (read-only)
`NSCLIENT-Report.ps1`	Fleet-wide deployment status for the client app + certificate coverage via Microsoft Graph (read-only)
`NSCLIENT-Remediation.ps1`	Safe remediation of failed/unhealthy clients; defaults to -WhatIf
`NSCLIENT-Set-RolloutRingTargeting.ps1`	Builds/previews the rollout-ring Entra groups for the ringed deployment

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the model: Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

The Netskope client on every device — healthy, enrolled, and steering correctly — deployed through Intune in safe rings, with an exception list that keeps business apps working and a rollback path proven before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

NSCLIENT-Discovery.ps1

Purpose: Read-only. Reports the OS build, whether the device is enrolled in Intune/MDM, and whether a prior or partial Netskope client is already present (service and/or install path). Use before a ringed rollout to spot devices that are not ready or that carry a leftover agent. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Discovery.ps1 EXPECTED RESULTS OS build, Intune/MDM enrollment state, and a flag if an existing Netskope client is detected. .NOTES Network Solutions — read-only. Netskope client specifics (exact service/install path) are vendor-version-dependent and deferred to the official Netskope documentation; the names probed below are common defaults and should be confirmed against your tenant's client build. #> [CmdletBinding()] param()

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Discovery.ps1 EXPECTED RESULTS OS build, Intune/MDM enrollment state, and a flag if an existing Netskope client is detected. .NOTES Network Solutions — read-only. Netskope client specifics (exact service/install path) are vendor-version-dependent and deferred to the official Netskope documentation; the names probed below are common defaults and should be confirmed against your tenant's client build. #> [CmdletBinding()] param()

NSCLIENT-Remediation.ps1

Purpose: Safely nudges a device back toward a healthy client: triggers an Intune/MDM sync so the client app and certificate re-apply, and (if the client service exists but is stopped) attempts to start it. Defaults to -WhatIf: it PREVIEWS actions and changes nothing until you pass -Apply. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Remediation.ps1 Apply changes: powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Remediation.ps1 -Apply EXPECTED RESULTS In preview: a list of what would be done. With -Apply: MDM sync requested and the client service started if it was stopped. Re-run NSCLIENT-Validation.ps1 after a policy cycle. .NOTES Network Solutions. Default is -WhatIf (preview). Reinstall/repair of the client itself is handled by re-pushing the Intune app; deep client repair is vendor-specific and DEFERRED to Netskope docs. #> [CmdletBinding()] param([switch]$Apply # without this, the script runs in -WhatIf preview mode and changes nothing) $WhatIf = -not $Apply

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Remediation.ps1 Apply changes: powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Remediation.ps1 -Apply EXPECTED RESULTS In preview: a list of what would be done. With -Apply: MDM sync requested and the client service started if it was stopped. Re-run NSCLIENT-Validation.ps1 after a policy cycle. .NOTES Network Solutions. Default is -WhatIf (preview). Reinstall/repair of the client itself is handled by re-pushing the Intune app; deep client repair is vendor-specific and DEFERRED to Netskope docs. #> [CmdletBinding()] param([switch]$Apply # without this, the script runs in -WhatIf preview mode and changes nothing) $WhatIf = -not $Apply

NSCLIENT-Report.ps1

Purpose: Read-only. Reports, via Microsoft Graph, the Intune deployment status of the Netskope client app across managed Windows devices (installed / failed / pending / not applicable), so you can see per-ring coverage and chase failures. Optionally writes a per-device CSV. HOW TO RUN powershell

-ExecutionPolicy Bypass -File .\NSCLIENT-Report.ps1 -AppName "Netskope Client - Windows - Production" -CsvPath C:\Temp\nsclient.csv Requires the Microsoft Graph PowerShell SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A status breakdown (installed/failed/pending/...) for the named app and an optional per-device CSV. .NOTES Network Solutions — read-only against Microsoft Graph. Certificate-profile coverage is reported separately in the Intune portal; this script focuses on the client app deployment state. Steering state is device-side / Netskope-side and is validated by NSCLIENT-Validation.ps1 (DEFERRED to docs). #> [CmdletBinding()] param([string]$AppName = 'Netskope Client - Windows - Production', [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Report.ps1 -AppName "Netskope Client - Windows - Production" -CsvPath C:\Temp\nsclient.csv Requires the Microsoft Graph PowerShell SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A status breakdown (installed/failed/pending/...) for the named app and an optional per-device CSV. .NOTES Network Solutions — read-only against Microsoft Graph. Certificate-profile coverage is reported separately in the Intune portal; this script focuses on the client app deployment state. Steering state is device-side / Netskope-side and is validated by NSCLIENT-Validation.ps1 (DEFERRED to docs). #> [CmdletBinding()] param([string]$AppName = 'Netskope Client - Windows - Production', [string]$CsvPath)

NSCLIENT-Set-RolloutRingTargeting.ps1

Purpose: Creates (or previews) the Entra security groups that the ringed Netskope client rollout is assigned to: Device Group - Pilot - Stage1, Device Group - Early - Stage2, Device Group - Broad - Stage3. Defaults to -WhatIf: it PREVIEWS and changes nothing unless you pass -Apply. It does not move members between rings; it ensures the ring groups exist. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Set-RolloutRingTargeting.ps1 Apply (create groups that are missing): powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Set-RolloutRingTargeting.ps1 -Apply Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All (preview needs only Group.Read.All). EXPECTED RESULTS A report of which ring groups exist; with -Apply, any missing ring group is created. .NOTES Network Solutions. Default is -WhatIf (preview). Assigning these groups to the client app and the certificate profile is done in the Intune portal per the Implementation Guide. #> [CmdletBinding()] param([switch]$Apply # without this, the script previews and creates nothing) $WhatIf = -not $Apply

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Set-RolloutRingTargeting.ps1 Apply (create groups that are missing): powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Set-RolloutRingTargeting.ps1 -Apply Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All (preview needs only Group.Read.All). EXPECTED RESULTS A report of which ring groups exist; with -Apply, any missing ring group is created. .NOTES Network Solutions. Default is -WhatIf (preview). Assigning these groups to the client app and the certificate profile is done in the Intune portal per the Implementation Guide. #> [CmdletBinding()] param([switch]$Apply # without this, the script previews and creates nothing) $WhatIf = -not $Apply

NSCLIENT-Validation.ps1

Purpose: Read-only. Validates device-side client health: the client is installed, its service is running, it is enrolled to a tenant, and steering appears active. Use at the pilot gate and after each ring expansion. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Validation.ps1 EXPECTED RESULTS "CLIENT HEALTHY" when installed + service running + enrolled + steering active; otherwise a per-check breakdown of what is missing. .NOTES Network Solutions — read-only. Netskope

client internals (exact service name, enrollment/steering state location) are vendor-version-dependent and DEFERRED to the official Netskope documentation; confirm the probes below against your client build. This script does not change client state. #> [CmdletBinding()] param()

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\NSCLIENT-Validation.ps1 EXPECTED RESULTS "CLIENT HEALTHY" when installed + service running + enrolled + steering active; otherwise a per-check breakdown of what is missing. .NOTES Network Solutions — read-only. Netskope client internals (exact service name, enrollment/steering state location) are vendor-version-dependent and DEFERRED to the official Netskope documentation; confirm the probes below against your client build. This script does not change client state. #> [CmdletBinding()] param()